KBTU, School of Information Technologies and Engineering
MSITM 5134, Management of Organizational Data

# Midterm Project 2026, Group 9 Topic Java Vulnerability Detection with CodeBERT and Class-Weighted Cross-Entropy

A partial reproduction of Shestov et al. (2025), with the data managed from download to decision

**Student:** Bekzat Saulebay (25MD0294), individual work on the Group 9 topic

**Lecturer:** Seralina Nazgul Tynybekkyzy

**Date:** 27 October 2026

**Data:** X1 dataset of Shestov et al., GitHub `rmusab/vul-llm-finetune`, commit `862ed17`

**Files:** `data_pipeline.py`, `train.py`, `alternative.py`, `baselines.py`, `evaluate.py`, `demo.py`, `midterm_project.ipynb`, `config.json`, `requirements.txt`, `README.md`

## 1. Task and what I did

The Group 9 topic is to detect vulnerable Java functions with CodeBERT and a class-weighted cross-entropy loss, as a partial reproduction of the study by Shestov et al. (2025). I did the whole project on my own. The task also asks to manage the data through its whole lifecycle and to show how governance, quality, integration, security and analytics affect the model. The report follows that order: one section for each lecture, then the model, the results, and what an organization should do with them.

The task table gives "CodeBERT + Class-Weighted Loss /" as the alternative method for Group 9, and the part after the slash is empty. I read it as the light version of the same idea: CodeBERT is frozen and only turns each function into a vector, and a small classifier is trained on these vectors with the same class weights. The primary method ran on my laptop without problems, so I did not have to switch. I ran the alternative anyway, because the project objective asks to compare the two on quality, cost and risk.

My research question comes from the paper. The authors write that plain class weighting did not work for their large model, so they moved on to focal loss and sample weights. I test whether class weighting helps CodeBERT, a model of the same family as ContraBERT, the smaller model the paper compares against, and what the answer means for an organization that wants to use such a model.

What I built:

- a data pipeline that downloads the dataset at a fixed commit, checks it, cleans it and loads it into a SQLite warehouse that follows my ER model;
- CodeBERT fine-tuned with class-weighted cross-entropy, two ablations (no weights, milder weights) and one run on the balanced version of the dataset;
- the alternative method and three simple reference models;
- an evaluation with 95% confidence intervals, charts and a live demo;
- governance and security rules that the code enforces, not only describes.

> **Main results**
>
> - CodeBERT with class-weighted cross-entropy reached a test ROC AUC of 0.849 and an F1 of 0.222, the same as the paper's CodeBERT-style baseline (0.85 and 0.22).
> - Class weighting did not help. The same model without weights was at least as good (ROC AUC 0.862, F1 0.278). The weights only move the decision threshold.
> - Accuracy depends on the threshold: 90.5% with the paper's rule, 93.1% at 0.5. A model that flags nothing reaches 97.4%, so the 93% target says little on this data.
> - No model can tell a vulnerable function from its own fixed version (ROC AUC about 0.5). The models learn which code a fix touched, and a simple TF-IDF model does as well as CodeBERT.
> - As a triage aid the model still helps: reading the top 10% of functions finds 53% of the vulnerable ones (63% without weights).

## 2. Data lifecycle and information value chain (Lecture 1)

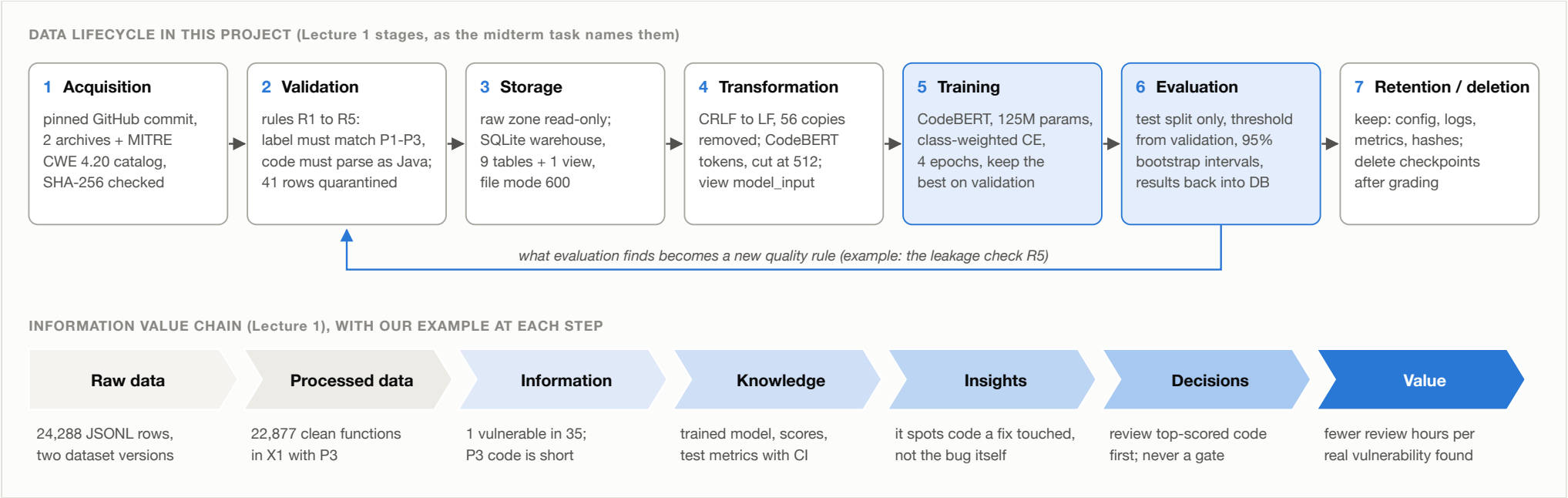



Figure 1. The data lifecycle of the project, with the stages named in the task, and the information value chain with my example at each step.

Table 1. Each stage mapped to its DAMA-DMBOK knowledge area and to a standard or framework from Lecture 1.

| Stage | DAMA-DMBOK knowledge area | Standard or framework | What it means in my project |
|---|---|---|---|
| Acquisition | Data Integration and Interoperability; Metadata Management | ISO 8000 (provenance); ISO/IEC 27001 (supplier integrity) | source commit, URL and SHA-256 recorded for every file I take in |
| Validation | Data Quality | ISO 8000; ISO/IEC 25012 quality model | rules R1 to R5, each with a fixed action; quarantine instead of guessing |
| Storage | Data Storage and Operations; Data Security | ISO/IEC 27001 | read-only raw zone, one warehouse file that only its owner can open |
| Transformation | Data Modeling and Design; Reference and Master Data | ISO/IEC 11179 (metadata) | the label comes from one reference table; CWE ids are checked against MITRE |
| Training | Data Science (the DMBOK chapter on big data and data science) | NIST AI RMF; ISO/IEC 42001 | one configuration file, fixed seed, pinned model revision, a summary for every run |
| Evaluation | Data Warehousing and Business Intelligence | NIST AI RMF ("measure") | metrics with confidence intervals, results stored next to the data they describe |
| Retention and deletion | Data Governance; Document and Content Management | ISO/IEC 38505 (governance of data) | a written rule for what I keep, what I delete and why |

Figure 1 shows the lifecycle. Two choices shape the rest of the project.

First, every input is pinned. The dataset comes from one commit of the authors' GitHub repository, the CWE catalog is version 4.20, and CodeBERT is one exact revision on Hugging Face. Each archive is checked against its SHA-256 on every run. If the authors change a file, the pipeline stops instead of quietly training on different data.

Second, the lifecycle is a loop. Problems found during evaluation become quality rules. In Lab 4 a model was tested on code it had already seen in training. That is why rule R5 now removes copies of training functions from the validation and test splits before any model runs.

Retention follows from pinning. The archives can be downloaded again from the pinned commit, so I keep their hashes for good and the files only while I work. Checkpoints take about 500 MB each. After grading I keep the primary one for the demo and delete the rest. Logs, configuration, metrics and hashes are small and stay.

The information value chain from Lecture 1 goes from raw data to processed data, information, knowledge, insights, decisions and value. The bottom row of Figure 1 gives my example for each step. A score has no value by itself. It has value when it changes a decision, here the order in which a security team reviews code. This is why section 9 measures how many real vulnerabilities a team finds for a given amount of review work, and not only accuracy.

## 3. Data governance (Lecture 2)

### Ownership and access

I do not own the source data. Its owners are the people who published it. I am the custodian of my copy and the owner of what I build from it.

| Data asset | Owner | My role | Licence and access |
|---|---|---|---|
| X1 archives, with and without P3 | dataset authors (GitHub `rmusab`) | data steward keeps the pinned copy | no licence file at the pinned commit (I checked the repository), so I use it for study only and do not redistribute the code |
| MITRE CWE catalog 4.20 | MITRE | reference data, read only | public; MITRE allows reuse if its copyright notice is kept |
| CodeBERT base model | Microsoft | ML engineer loads the pinned revision | MIT licence on Microsoft's CodeBERT GitHub repository; the Hugging Face page names no licence |
| Warehouse, clean data, quality log | me (the student) | data steward | only me |
| Fine-tuned models and predictions | me (the student) | ML engineer | me and the lecturer; weights are not uploaded |

### Responsibilities (RACI)

I used the RACI format and the five roles from Lecture 2. In a company these are different people. In this project I did all five jobs myself, so the table shows how the work would be shared in a real team.

| Task | Data owner | Data steward | Data / ML engineer | Compliance | Security |
|---|---|---|---|---|---|
| Approve the use of the dataset and the model (purpose, licence) | A | C | I | R | C |
| Pin sources and check their SHA-256 | I | A | R | I | C |
| Run the quality rules and review quarantined labels | C | A | R | I | C |
| Train and evaluate models, keep a summary of every run | I | C | A/R | I | C |
| Set the threshold and approve use in code review | A | I | C | I | R |
| Check personal data and legal exposure (commit authors, data location) | I | C | I | A | R |
| Keep and delete checkpoints and data copies | A | R | C | C | I |

R = responsible (does it), A = accountable (owns the result), C = consulted, I = informed.

### Usage policy

1. **Purpose.** The data and the models are used only for this course, to study vulnerability detection.
2. **No redistribution.** The source repository has no licence, so my submission does not contain the code samples. It pins the commit, and the pipeline downloads the files.
3. **A triage aid, never a gate.** No code is called safe because the model did not flag it.
4. **No analysis of people.** Commit ids lead to the developers who wrote the code. I never look up authors and never rank developers by their bugs.
5. **Labels are not edited by hand.** Conflicting labels go to quarantine with the reason and wait for a security expert.
6. **No model without a record.** Every model has a run summary with the data version, configuration, training time and SHA-256 of its weights. A model without one is not used.

### Ethics and law

- **Labels are noisy.** They come from a rule: the function before a fix is vulnerable, the same function after the fix is not. A fix can also change unrelated code, and a "fixed" function can still be vulnerable. Croft et al. (2023) found such errors in many vulnerability datasets. Even a perfect score would not mean a perfect detector.
- **Dual use.** A model that finds vulnerabilities helps an attacker as much as a defender. I only use public vulnerabilities that were fixed long ago, and I treat any list of predicted vulnerable functions as restricted data (section 7).
- **False confidence.** A model with 97% accuracy sounds safe to a manager. Section 9 shows why that number says almost nothing on this data.
- **Personal data.** A commit hash is not a name, but it leads to the author's name and email in Git. Under GDPR and the Kazakhstan Law No. 94-V on personal data, that makes it indirectly identifying data. I keep commit ids only because the split depends on them, and I never fetch anything about the authors. If a company ran this model on its own code, the results would point at its own staff, and the Kazakh law would also require that this data stays on servers in Kazakhstan.

## 4. Data model (Lecture 3)

Figure 2, on the first landscape page after section 5, is the conceptual model. I built the warehouse straight from it: every entity is a table with the same keys, and SQLite enforces the foreign keys. The model has two parts, the data (blue) and the experiments (orange), and they meet only in `prediction`.

- **The label is stored once.** The source files carry both `target` and `status`, but the status already decides the label: P1 (vulnerable) means 1, P2 (fixed) and P3 (unchanged) mean 0. Keeping both in the function table is a transitive dependency. It breaks third normal form and lets the two disagree. So the label lives in `label_partition`, and the view `model_input` joins it back for training.
- **A commit is an entity.** The authors split the data by commit, and a commit links the vulnerable and the fixed version of a function. The 839 commit hashes come in 8 different lengths (Lab 3), so I never join on them across sources. I use a fingerprint of the code for that.
- **Business rules are constraints.** A split is train, valid or test. A label is 0 or 1. A review status is clean, needs_expert_review or duplicate. A probability is between 0 and 1. A SHA-256 has 64 characters. The database refuses any row that breaks these rules.
- **Every row has lineage.** `source_file` and `source_line` point to the exact line in the original JSONL file, and every change made to a row is in `quality_issue`.

## 5. ETL and integration into the model workflow (Lecture 4)

Figure 3, on the second landscape page after this section, shows the whole system. Data moves from left to right, and evaluation writes its results back into the warehouse.

**Extract.** `data_pipeline.py` downloads the two archives and the CWE catalog, checks the SHA-256 of each file, unpacks the archives and makes the raw files read-only. It reads the six JSONL files into one table and adds the source file and line to every row: 24,288 rows.

**Transform.** The cleaning rules are in section 6. The pipeline also counts CodeBERT tokens for every function, so I know before training how much code the 512-token limit cuts off.

**Load.** The pipeline rebuilds the warehouse from scratch on every run. A full reload takes a few seconds and is simpler than an incremental load, because the source is a fixed snapshot.

Table 2. What the load step puts into the warehouse.

| Table | What it holds | Rows |
|---|---|---|
| `java_function` | every function from both versions, with lineage and review status | 24,288 |
| `quality_issue` | one row for every rule that fired on a function | 756 |
| `cwe` | MITRE CWE 4.20 weaknesses, categories and views, plus the two NVD placeholders | 1,452 |
| `vcs_commit` | fix commits | 839 |
| `label_partition` | P1, P2, P3 and the label each one means | 3 |
| `dataset_version` | the two archives with URL, SHA-256 and licence | 2 |
| `experiment_run`, `prediction`, `metric` | written by `evaluate.py` after training | see section 9 |

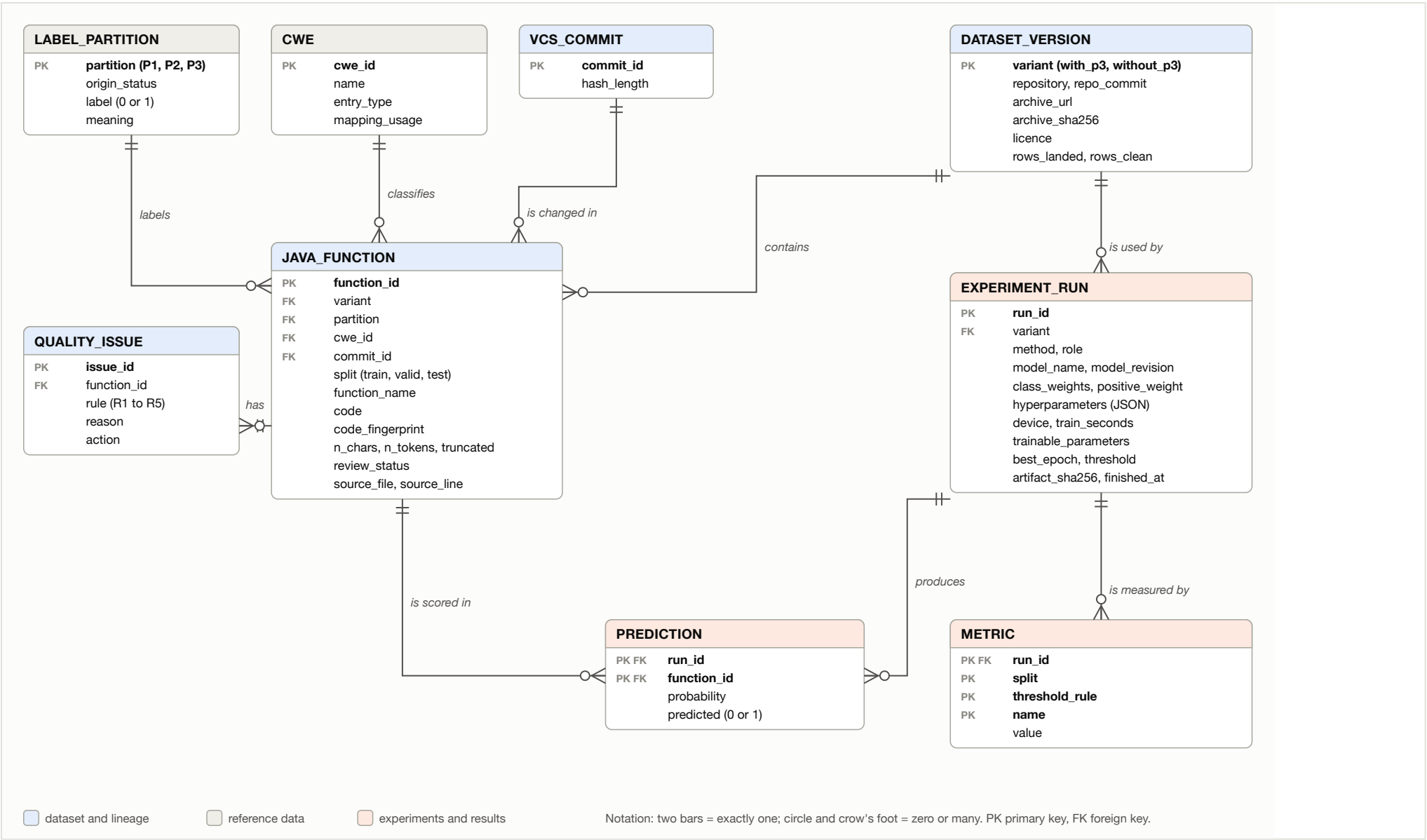



Figure 2. ER model of the dataset, the Java functions, the vulnerability labels and the experiment results (section 4). The warehouse has exactly these tables, keys and foreign keys.

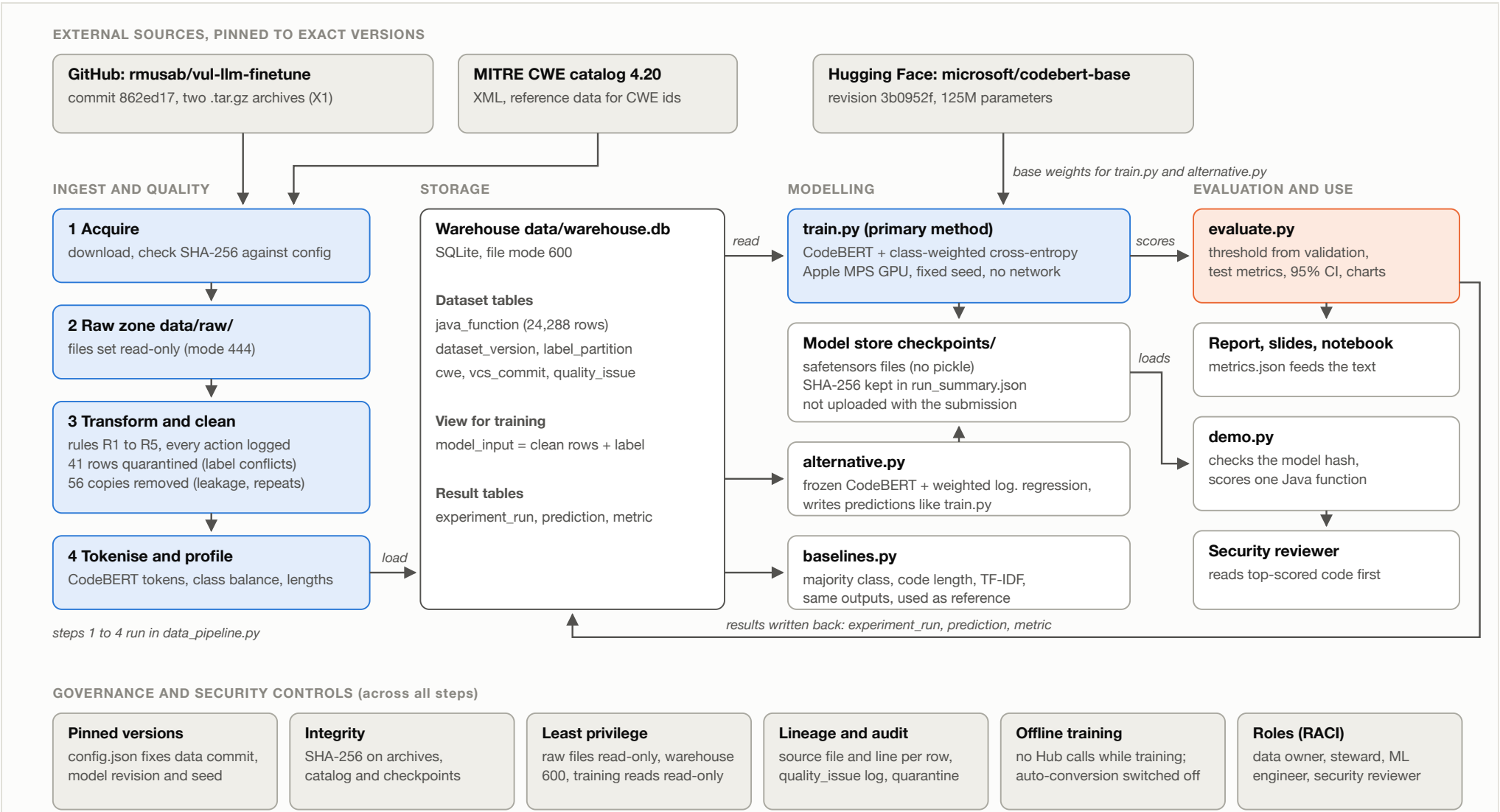



Figure 3. System architecture (section 5). Data moves from left to right; evaluation writes its results back into the warehouse. The bottom row lists the controls that apply to every step.

**Integration with the model.** The models do not read files. `train.py` gets its data with one SQL query on the view `model_input`, which only returns clean rows. A quarantined or duplicate row cannot reach training by mistake. After training, `evaluate.py` writes every run, every prediction and every metric back into the warehouse. The table of recall by CWE in section 9 is a SQL join of predictions, functions and the MITRE catalog.

## 6. Data quality (Lecture 5)

At first sight the data is clean: no missing values, every label is valid, every function parses as Java. The real problems are about meaning, not format. Table 3 lists my rules. I never change a label and never guess one. A row with a doubtful label goes to quarantine with its reason and stays in the warehouse for an expert.

Table 3. Cleaning rules and how many rows each one touched (both dataset versions together).

| Rule | Check | Action | Rows |
|---|---|---|---|
| R1 | Windows line endings | changed to Unix line endings, row kept | 659 |
| R2 | label, status or split not allowed or not consistent | quarantine | 0 |
| R3 | code is empty or does not parse as a Java method | quarantine | 0 |
| R4 | same function has another label in the other dataset version | quarantine, label kept as it was | 38 |
| R4 | identical code carries both labels | quarantine, label kept as it was | 3 |
| R5 | copy of a function that is already in an earlier split (leakage) | removed from the later split | 33 |
| R5 | copy repeated inside the same split | removed, first copy kept | 23 |

Three results matter. 19 functions have the label "vulnerable" in one version of the dataset and "not vulnerable" in the other, and 3 more rows have identical code with both labels. 33 rows in validation or test were copies of a function already in an earlier split, so a model would have been tested on code it had seen; I removed the later copy. And 659 functions had Windows line endings, which I changed to the usual Unix ones.

Table 4. Quality dimensions before and after cleaning.

| Dimension | With P3, before | With P3, after | Without P3, before | Without P3, after |
|---|---|---|---|---|
| Rows | 22,954 | 22,877 | 1,334 | 1,314 |
| Completeness | 100.00% | 100.00% | 100.00% | 100.00% |
| Uniqueness of code | 99.52% | 100.00% | 99.85% | 100.00% |
| Validity (values, Java syntax) | 100.00% | 100.00% | 100.00% | 100.00% |
| Label consistency | 99.90% | 100.00% | 98.58% | 100.00% |
| Functions in more than one split | 29 | 0 | 1 | 0 |
| Rows with Windows line endings | 624 | 0 | 35 | 0 |
| Share of vulnerable functions | 2.81% | 2.82% | 49.85% | 49.09% |
| Not vulnerable per vulnerable | 34.5 | 34.5 | 1.0 | 1.0 |

### Class distribution

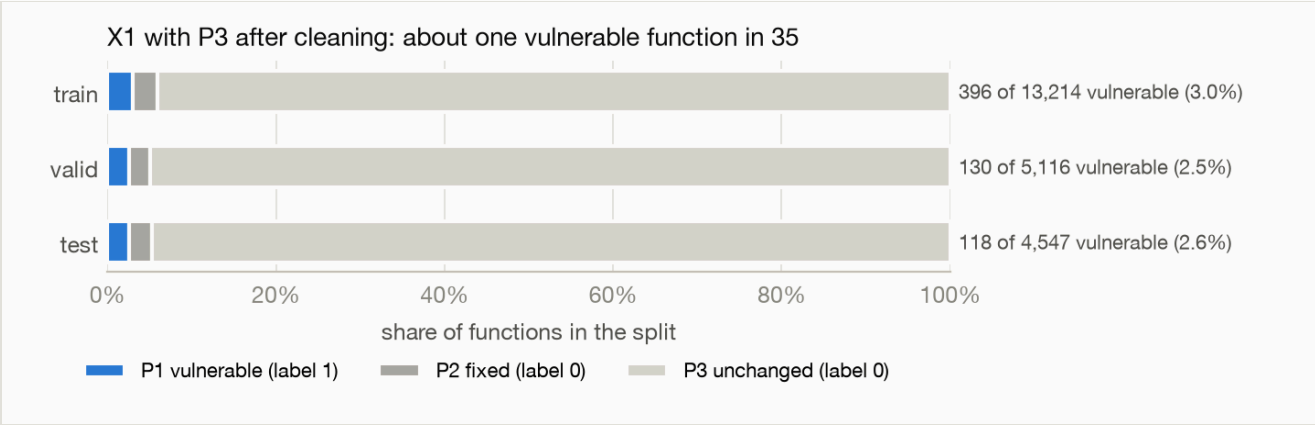



Figure 4. Share of vulnerable (P1), fixed (P2) and unchanged (P3) functions in each split of X1 with P3, after cleaning.

After cleaning, X1 with P3 has 644 vulnerable functions out of 22,877, about one in 35. The test split has only 118 of them. This has two consequences. A model that always says "not vulnerable" is right 97.4% of the time on the test split. And every recall or F1 on the test split rests on about a hundred cases, so I give confidence intervals. The balanced version (X1 without P3) has 1,314 functions, about half of them vulnerable.

### Length of the code

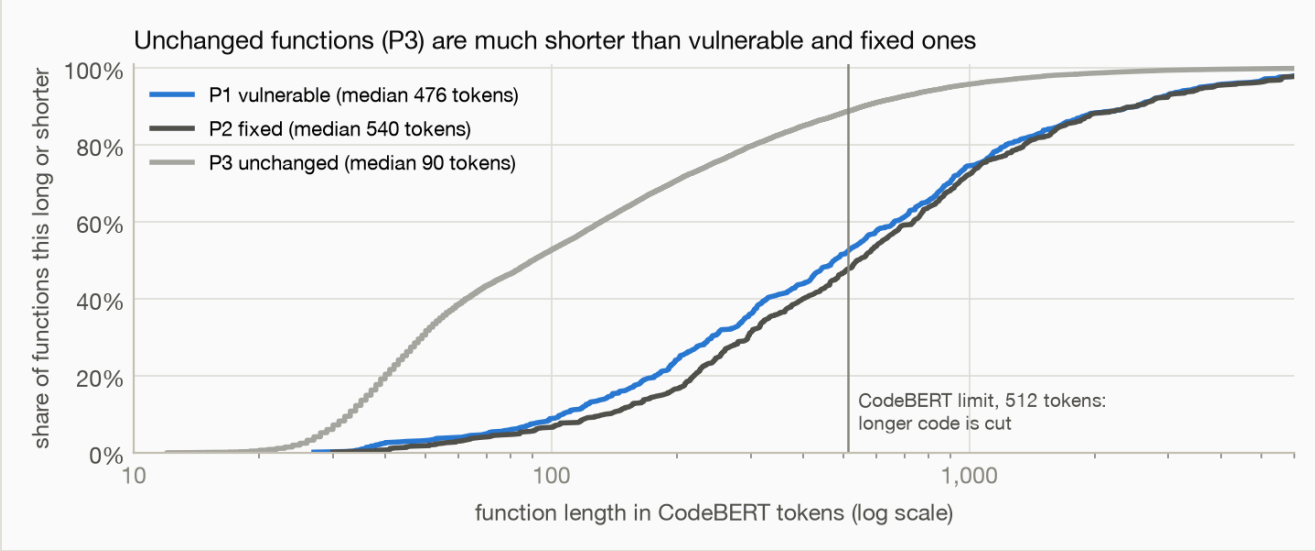



Figure 5. Length of each function in CodeBERT tokens, by partition (X1 with P3, after cleaning).

Vulnerable and fixed functions are long. Their median lengths are 476 and 540 tokens, and about half of them are longer than CodeBERT's limit of 512 tokens, so their end is cut off. The unchanged P3 functions have a median of only 90 tokens. This matters in two ways. The model may never see the line with the bug. And length alone separates P3 from P1 quite well, so a model can score high by learning that long code is risky. The length-only model in section 9 measures how far that gets.

## 7. Data security (Lecture 6)

The dataset is public, but the project still has assets to protect. In a company, the same pipeline would run on the company's own code, and its output would be a list of the places where that code is weak.

| Asset | Level | Why |
|---|---|---|
| Pinned archives, CWE catalog | Public | published by their owners |
| Warehouse, clean data, quality log | Internal | a copy of public code, but with my labels and decisions, and no licence to share the code |
| Configuration, logs, metrics, report | Internal | shared with the lecturer |
| Fine-tuned weights | Internal (Confidential in a company) | weights trained on private code can leak parts of that code |
| Predictions on a company's own code | Restricted | a ranked list of likely vulnerable functions is a map for an attacker |

### Controls I built in

| Control | How | Where to see it |
|---|---|---|
| Integrity of inputs | SHA-256 of both archives and the CWE catalog is checked on every run; a mismatch stops the pipeline | `config.json`, `data_pipeline.download` |
| Read-only raw zone | raw files are set to mode 444 right after download | notebook, section 6 |
| Least privilege | the warehouse file has mode 600; training, the alternative and the demo open it read-only | notebook, section 6: a DELETE on the read-only connection is refused |
| Safe model loading | CodeBERT is published as a pickle `.bin` file. `transformers` loads it with `weights_only=True` and refuses torch older than 2.6 because of CVE-2025-32434. I use torch 2.14.1 and a pinned revision | `requirements.txt`, `config.json` |
| My own models | saved as safetensors, which cannot hold code; the SHA-256 goes into `run_summary.json`, and `demo.py` refuses a file whose hash differs | `demo.py`, notebook section 6 |
| No hidden network calls | see the finding below | `common.py` |
| No secrets in code | no tokens or passwords in code or configuration; no Hugging Face token is needed | all files |

> **A library that calls home.** During a test run I saw a warning that my training script was sending requests to the Hugging Face Hub, although the model was already in the local cache. I looked in the `transformers` source: when it loads a `.bin` checkpoint, it starts a background thread that asks the Hub for a safetensors copy and can ask Hugging Face's conversion service to make one. For a company that trains on private code, a library that contacts an outside service on its own is a real risk. I switched it off (`DISABLE_SAFETENSORS_CONVERSION=1`), turned off telemetry and load models from the local cache only. After the change the warning was gone. I did not check the traffic with a network monitor.

### Risks

| Risk | What happens | Likelihood / impact | Control |
|---|---|---|---|
| The dataset changes upstream | a different model, and nobody notices | medium / high | pinned commit and SHA-256 |
| Wrong or poisoned labels | the model learns the wrong thing | medium / medium | quarantine, audit log, labels never edited by hand |
| A malicious model file | code runs on my machine when the model loads | low / high | pinned revision, `weights_only`, safetensors for my own files |
| A library contacts outside services | code or metadata leaves the machine | medium / high for private code | conversion and telemetry off, cache only |
| The prediction list leaks | attackers learn where the weak spots are | low / high | Restricted level, security team only |
| People trust the model too much | missed vulnerabilities reach production | high / high | triage aid only; recall reported next to accuracy |
| Results are used against developers | unfair blame, misuse of personal data | low / medium | usage policy rule 4, no author lookups |

## 8. Method and experiments

### Primary method: CodeBERT with class-weighted cross-entropy

CodeBERT (Feng et al., 2020) is a RoBERTa encoder with 125 million parameters, pretrained on source code and its documentation. I put the standard classification head on top (a small two-layer network on the first token) and fine-tune all weights.

Plain cross-entropy treats every function the same. With one vulnerable function in 35, a model can keep the loss low by mostly saying "not vulnerable". Class-weighted cross-entropy multiplies the loss of each example by the weight of its class:

$$L = -\Sigma_i \, w_{y_i} \log p(y_i \mid x_i) \,/\, \Sigma_i \, w_{y_i}, \quad w_c = N \,/\, (2\, N_c)$$

N is the number of training functions and $N_c$ the number in class c. The training split has 13,214 functions, 396 of them vulnerable, so the weights are 0.515 for "not vulnerable" and 16.68 for "vulnerable". One vulnerable function counts as much as 32 others. The milder ablation uses the square root of these weights (0.718 and 4.08).

### Protocol

I follow the protocol of the paper (appendix B): train for a fixed number of epochs, keep the checkpoint with the best ROC AUC on the validation split, choose the threshold that gives the best F1 on the validation split, and only then score the test split, once. The test split plays no part in any choice.

Table 5. Model configuration (the full file is `config.json`).

| Setting | Value |
|---|---|
| Base model | `microsoft/codebert-base`, revision `3b0952f` |
| Head | RobertaForSequenceClassification, 2 classes |
| Input | code only, cut at 512 tokens; batches grouped by length, padded per batch |
| Optimizer | AdamW, learning rate 2e-05, weight decay 0.01, linear warm-up over 10% of steps, then linear decay |
| Batch size | 16 for training, 64 for scoring |
| Gradient clipping | norm 1.0 |
| Epochs | 4 on X1 with P3, 10 on X1 without P3; the best epoch on validation is kept |
| Seed, device | 42, Apple M3 Pro GPU (MPS), 32-bit floats |

### Alternative method and reference models

**Alternative.** CodeBERT is frozen. One pass over the data turns every function into a 768-number vector (the mean of CodeBERT's last layer over all tokens). A logistic regression with the same balanced class weights is trained on these vectors; its strength C is chosen on the validation split. Only 769 numbers are trained instead of 124,647,170.

**Reference models.** Three simple models show what the numbers mean. "Always not vulnerable" shows what accuracy is worth on this data. "Code length only" is a logistic regression on the length of the function and nothing else. "TF-IDF" is the class-weighted logistic regression on code tokens from Lab 4.

Table 6. All runs. Class weights are given as "not vulnerable / vulnerable". Times are on the M3 Pro GPU.

| Run | Role | Data | Class weights | Best epoch | Trained parameters | Time |
|---|---|---|---|---|---|---|
| codebert_weighted | primary method | with P3 | balanced (0.515 / 16.68) | 4 of 4 | 124,647,170 | 49 min |
| codebert_plain | ablation, paper baseline without weights | with P3 | none (1.000 / 1.00) | 4 of 4 | 124,647,170 | 48 min |
| codebert_sqrt | ablation, milder weights | with P3 | square root of balanced (0.718 / 4.08) | 4 of 4 | 124,647,170 | 48 min |
| codebert_weighted_balanced_set | same method on the balanced dataset | without P3 | balanced (0.984 / 1.02) | 4 of 10 | 124,647,170 | 14 min |
| frozen_codebert_lr | alternative method | with P3 | balanced (0.515 / 16.68) | – | 769 | 3 min |
| tfidf_lr_with_p3 | reference: classic text model (Lab 4 baseline) | with P3 | balanced | – | 100,001 | 2 s |
| length_lr_with_p3 | reference: how far a model gets on length alone | with P3 | balanced | – | 2 | 0 s |
| majority_with_p3 | reference: what accuracy looks like with no model | with P3 | balanced | – | – | 0 s |

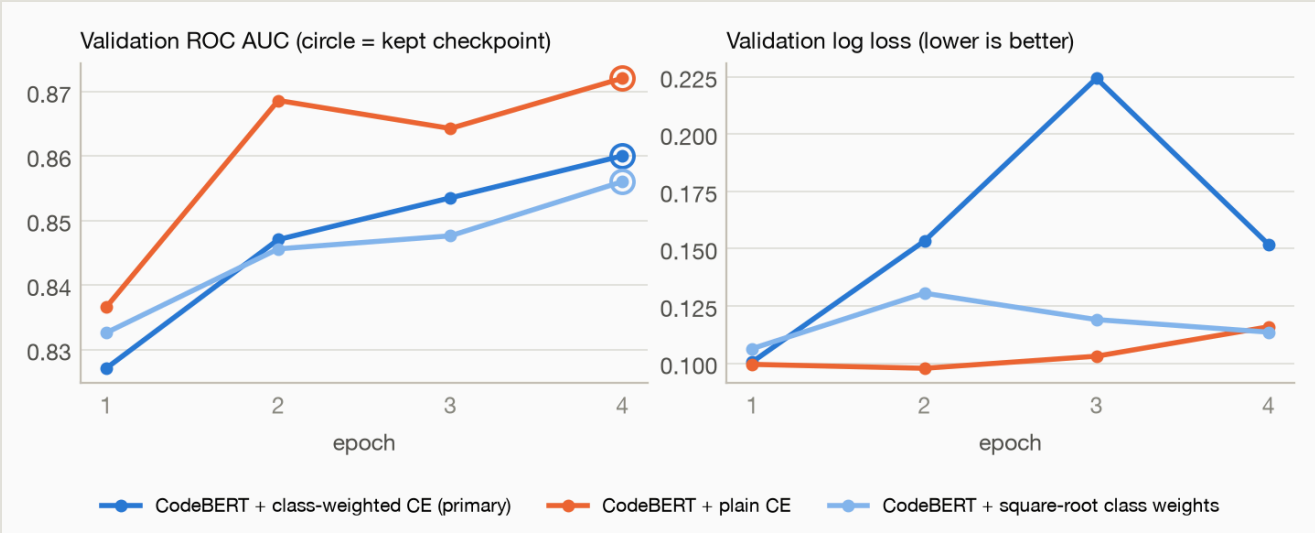



Figure 6. Validation ROC AUC and log loss after each epoch (X1 with P3). The circled point is the checkpoint I kept. Full logs: `runs/<run>/train.log`.

## 9. Results and what they mean (Lecture 7)

### 9.1 The test results

Table 7 shows every model on the test split of X1 with P3. The primary model reaches a ROC AUC of 0.849 (95% interval 0.816 to 0.880) and an F1 of 0.222. The paper reports 0.85 and 0.22 for ContraBERT, the CodeBERT-style model it compares against, so my partial reproduction lands on the same numbers.

Table 7. Test split of X1 with P3 (4,547 functions, 118 vulnerable). Threshold chosen on validation for the best F1. In brackets: 95% bootstrap interval.

| Model | Accuracy | Precision | Recall | F1 | ROC AUC | PR AUC | ROC AUC, P1 vs P2 only |
|---|---|---|---|---|---|---|---|
| CodeBERT + class-weighted CE (primary) | 90.5% | 0.141 | 0.525 | 0.222 (0.177–0.270) | 0.849 (0.816–0.880) | 0.156 | 0.470 |
| CodeBERT + plain CE | 92.8% | 0.188 | 0.534 | 0.278 (0.226–0.330) | 0.862 (0.828–0.894) | 0.186 | 0.485 |
| CodeBERT + square-root weights | 92.1% | 0.145 | 0.415 | 0.215 (0.165–0.264) | 0.851 (0.820–0.880) | 0.142 | 0.473 |
| Frozen CodeBERT + weighted LR (alternative) | 86.6% | 0.091 | 0.466 | 0.153 (0.118–0.190) | 0.819 (0.784–0.853) | 0.116 | 0.460 |
| TF-IDF + weighted LR | 95.5% | 0.268 | 0.415 | 0.326 (0.255–0.400) | 0.871 (0.839–0.901) | 0.214 | 0.505 |
| Code length only | 89.9% | 0.105 | 0.381 | 0.165 (0.124–0.206) | 0.807 (0.772–0.841) | 0.135 | 0.470 |
| Always "not vulnerable" | 97.4% | 0.000 | 0.000 | 0.000 (0.000–0.000) | 0.500 (0.500–0.500) | 0.026 | – |

The surprise is the simple models. TF-IDF with a logistic regression, the Lab 4 baseline, reaches a ROC AUC of 0.871, and its F1 (0.326) and PR AUC are clearly better than CodeBERT's (Table 9). A model that sees nothing but the length of the code reaches 0.807. On this benchmark, a transformer with 125 million parameters does not beat a bag of words. Section 9.4 shows why.

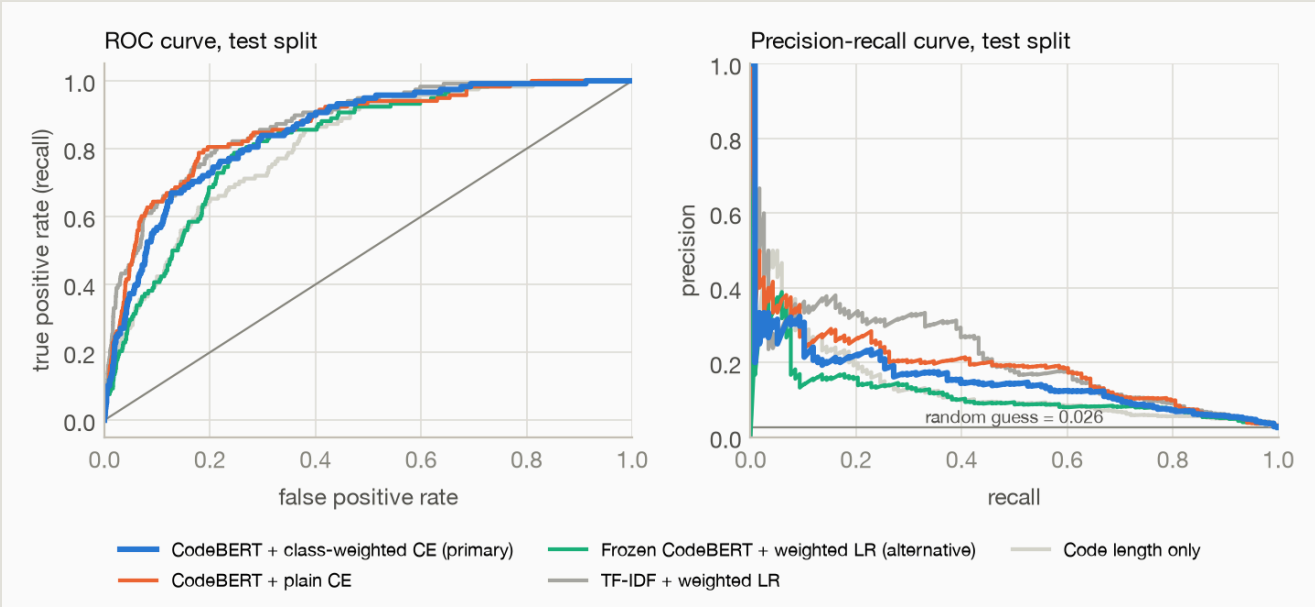



Figure 7. ROC and precision-recall curves on the test split. The ROC curves look strong; the precision-recall curves show how few flagged functions are really vulnerable.

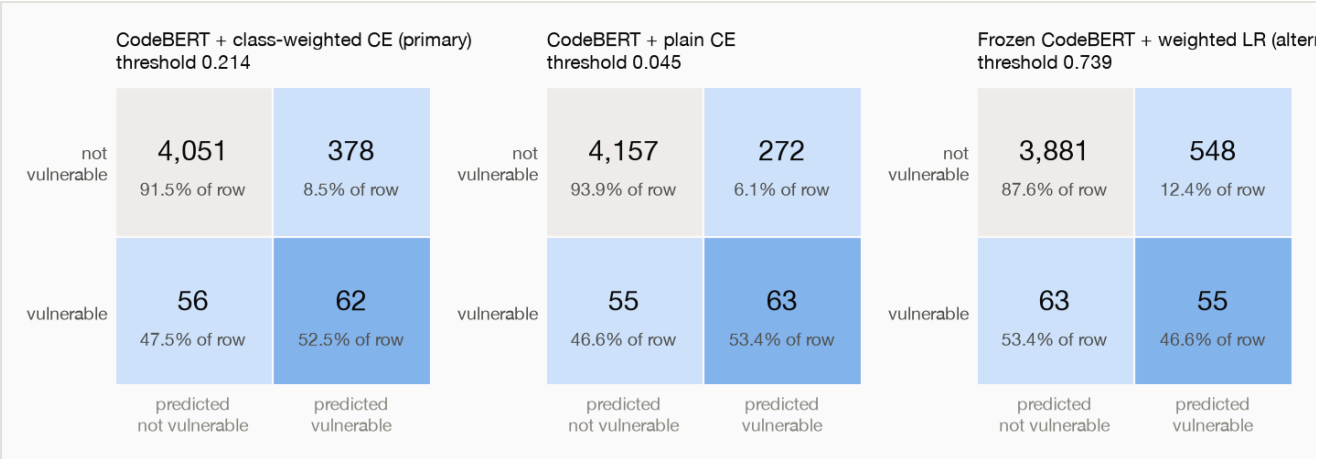



Figure 8. Confusion matrices on the test split, at the threshold each model chose on validation.

## 9.2 The accuracy target

The task asks for an accuracy of at least 93%. On this data the target says little, because the model that never flags anything reaches 97.4%. What a real model reaches depends on its threshold, so Table 8 gives three operating points, each fixed before the test split was scored.

Table 8. Three ways to set the threshold, scored on the test split. Only the fixed 0.5 rule does not use validation.

| Model | Threshold rule | Threshold | Flagged | Accuracy | Recall | 93% met |
|---|---|---|---|---|---|---|
| CodeBERT + class-weighted CE (primary) | best F1 on validation (paper) | 0.214 | 440 | 90.5% | 0.525 | no |
| | 93% accuracy on validation | 0.232 | 427 | 90.7% | 0.525 | no |
| | fixed 0.5 | 0.500 | 286 | 93.1% | 0.373 | yes |
| CodeBERT + plain CE | best F1 on validation (paper) | 0.045 | 335 | 92.8% | 0.534 | no |
| | 93% accuracy on validation | 0.032 | 391 | 91.9% | 0.602 | no |
| | fixed 0.5 | 0.500 | 8 | 97.4% | 0.025 | yes |
| CodeBERT + square-root weights | best F1 on validation (paper) | 0.164 | 338 | 92.1% | 0.415 | no |
| | 93% accuracy on validation | 0.063 | 448 | 90.3% | 0.534 | no |
| | fixed 0.5 | 0.500 | 134 | 95.6% | 0.220 | yes |
| Frozen CodeBERT + weighted LR (alternative) | best F1 on validation (paper) | 0.739 | 603 | 86.6% | 0.466 | no |
| | 93% accuracy on validation | 0.801 | 443 | 89.6% | 0.381 | no |
| | fixed 0.5 | 0.500 | 1,224 | 74.6% | 0.788 | no |
| TF-IDF + weighted LR | best F1 on validation (paper) | 0.662 | 183 | 95.5% | 0.415 | yes |
| | 93% accuracy on validation | 0.528 | 382 | 91.8% | 0.542 | no |
| | fixed 0.5 | 0.500 | 429 | 91.1% | 0.610 | no |

With the paper's rule (best F1 on validation) the model flags 440 of the 4,547 test functions, and its accuracy is 90.5%, below the target. When I pick the threshold that gives 93% on validation instead, test accuracy is still only 90.7%. The reason is a shift between the splits: at the same threshold the model flags 6.8% of the validation functions but 9.7% of the test functions, because the test commits come from other code. At the default threshold 0.5 the model flags 286 functions and reaches 93.1%, so the target is met, but recall drops from 0.525 to 0.373.

> **The 93% target can be met, but only by flagging less.** On this data accuracy mostly measures how few functions a model flags. For a security team the useful question is how many real vulnerabilities it finds for the review work it can afford (section 9.5).

### 9.3 Does class weighting help?

This was my research question, and the answer is no. Table 9 compares the two runs on the same 2,000 resamples of the test split. The model trained without weights ranks the functions at least as well: its ROC AUC is 0.862 against 0.849, and its PR AUC 0.186 against 0.156. Neither difference is clear, because both intervals include zero. Its F1 at the tuned threshold is clearly higher (0.278 against 0.222), and it finds more vulnerable functions at every review budget (Table 10). It was also better on validation after every epoch (Figure 6).

What the weights do is move the scores. Without weights the model gives low scores to almost everything: at the default threshold 0.5 it flags only 8 test functions. With weights it flags 286. The paper's protocol picks the threshold on validation anyway, so this shift buys nothing; the plain model simply gets a lower threshold (0.045 against 0.214). The weights also make the probabilities less honest: the plain model's validation log loss stays near 0.10, while the weighted one rises to 0.22 after the third epoch. The milder square-root weights (0.718 and 4.08) change nothing here: test ROC AUC 0.851 and F1 0.215, with no clear difference from the full weights (Table 9). More weight never helped, and no weight did best.

**Class weighting did not help CodeBERT, the same answer the paper gives for its large model.** The weights act like a lower threshold built into training. Choosing the threshold on validation gives the same effect without retraining, and the model trained without weights was at least as good on every measure.

Table 9. Paired bootstrap on the test split: the same 2,000 resamples for both models. "Clear" means the 95% interval of the difference does not include zero.

| Comparison | Metric | Difference | 95% interval | Clear |
|---|---|---|---|---|
| primary minus CodeBERT + plain CE | ROC AUC | -0.013 | -0.028 to +0.003 | no |
| primary minus CodeBERT + plain CE | PR AUC | -0.030 | -0.069 to +0.001 | no |
| primary minus CodeBERT + plain CE | F1 | -0.056 | -0.084 to -0.028 | yes |
| primary minus CodeBERT + square-root weights | ROC AUC | -0.002 | -0.012 to +0.008 | no |
| primary minus CodeBERT + square-root weights | PR AUC | +0.014 | -0.018 to +0.045 | no |
| primary minus CodeBERT + square-root weights | F1 | +0.007 | -0.023 to +0.041 | no |
| primary minus Frozen CodeBERT + weighted LR (alternative) | ROC AUC | +0.030 | +0.008 to +0.053 | yes |
| primary minus Frozen CodeBERT + weighted LR (alternative) | PR AUC | +0.040 | -0.013 to +0.089 | no |
| primary minus Frozen CodeBERT + weighted LR (alternative) | F1 | +0.070 | +0.036 to +0.103 | yes |
| primary minus TF-IDF + weighted LR | ROC AUC | -0.022 | -0.048 to +0.002 | no |
| primary minus TF-IDF + weighted LR | PR AUC | -0.059 | -0.124 to -0.002 | yes |
| primary minus TF-IDF + weighted LR | F1 | -0.103 | -0.161 to -0.044 | yes |
| primary minus Code length only | ROC AUC | +0.042 | +0.022 to +0.064 | yes |
| primary minus Code length only | PR AUC | +0.021 | -0.046 to +0.082 | no |
| primary minus Code length only | F1 | +0.058 | +0.024 to +0.096 | yes |

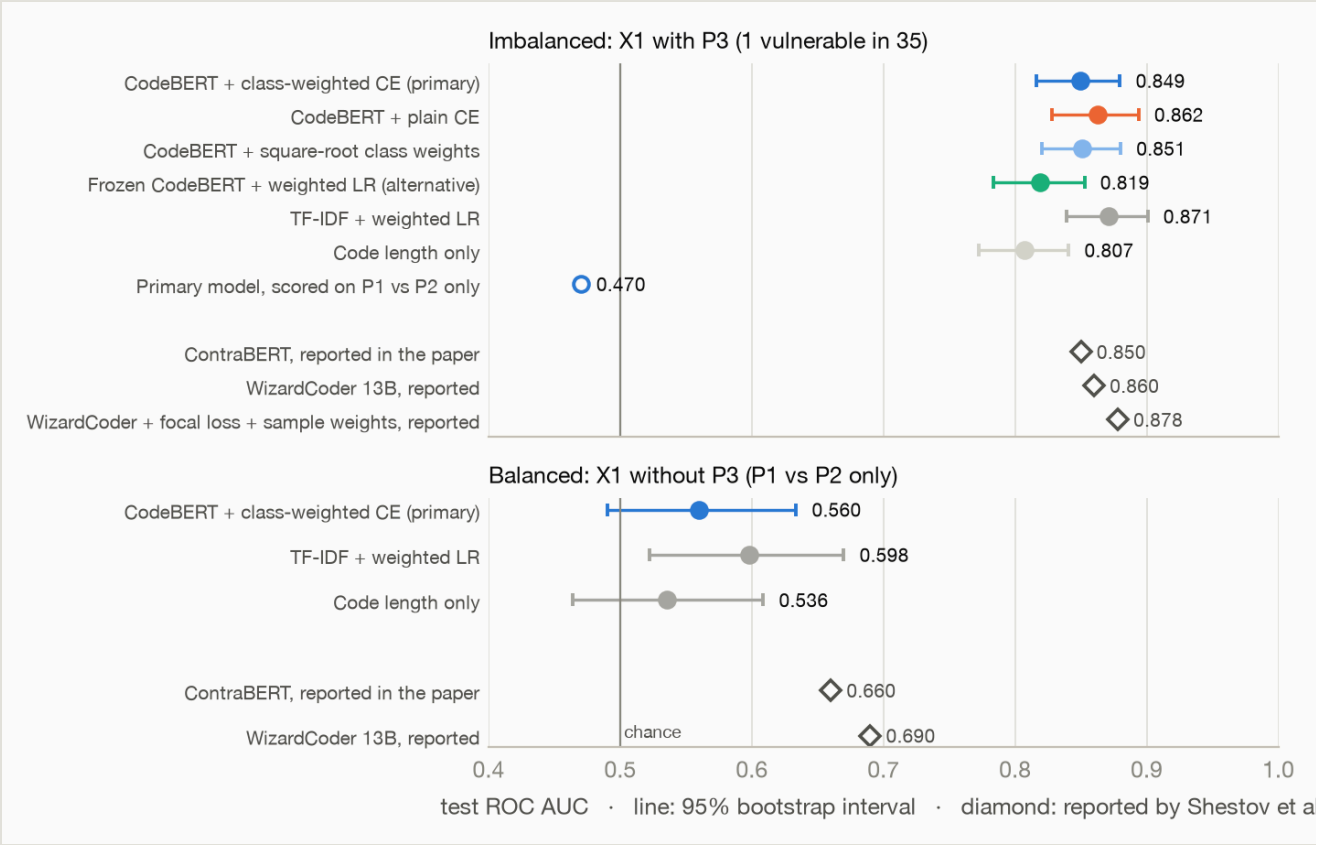



Figure 9. Test ROC AUC of every model with its 95% bootstrap interval, next to the numbers reported by Shestov et al. (diamonds).

## 9.4 What the model actually learned

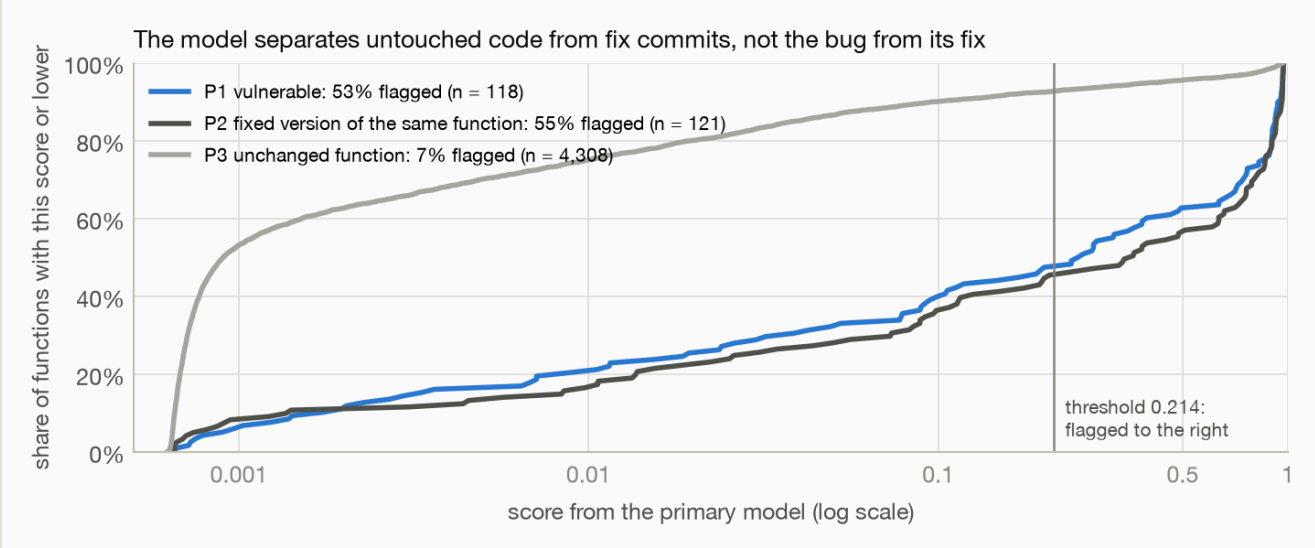



Figure 10. Scores of the primary model on the test split, by partition. The vulnerable functions (blue) and their fixed versions (dark gray) get almost the same scores.

Figure 10 splits the test functions by partition. At its threshold the model flags 53% of the vulnerable functions and 55% of their fixed versions, but only 7% of the unchanged functions. On vulnerable functions against their own fixed versions its ROC AUC is 0.470, which is chance. TF-IDF and the length model are at chance there too (last column of Table 7).

So the model has learned to tell code that a fix commit touched from code that no fix touched. It has not learned what makes code vulnerable. This fits what section 6 found: unchanged functions are short, and a vulnerable function and its fix differ in a few tokens, often after the 512-token cut. Most of the high ROC AUC on X1 with P3 comes from the easy P3 negatives. The paper shows the same from the other side: on the balanced dataset, where every negative is a fixed function, its best model reaches only 0.69.

### 9.5 What a review team gets

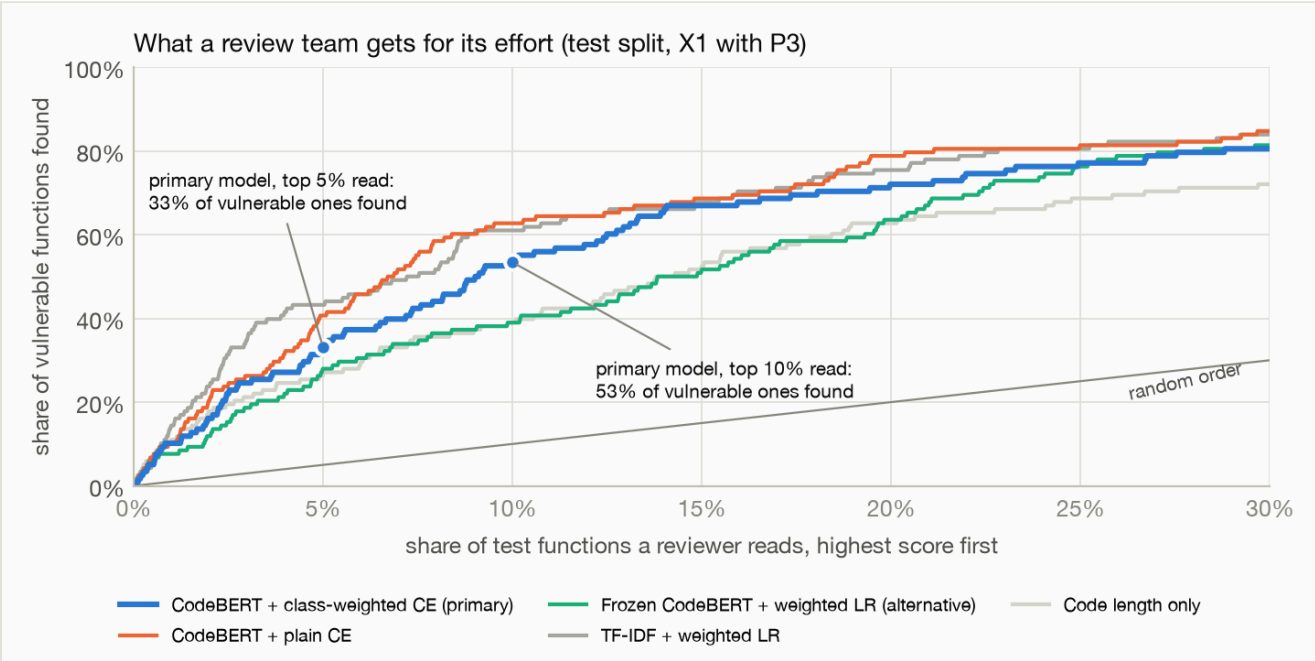



Figure 11. Share of vulnerable test functions found when a reviewer reads the functions in score order.

A model like this is useful as a way to order work. If a team reads the top 5% of test functions by score (227 functions), it finds 33% of the vulnerable ones. Reading 10% finds 53%, and reading 20% finds 72%. In random order the team would find 5%, 10% and 20%. TF-IDF does a little better at every budget (Table 10).

Table 10. Vulnerable functions found when a reviewer reads the top-scored share of the test split (118 vulnerable in total). Random order would find the same share as the share read.

| Model | top 1% (45 functions) | top 2% (91 functions) | top 5% (227 functions) | top 10% (455 functions) | top 20% (909 functions) |
|---|---|---|---|---|---|
| CodeBERT + class-weighted CE (primary) | 12 (10%) | 19 (16%) | 39 (33%) | 63 (53%) | 85 (72%) |
| CodeBERT + plain CE | 12 (10%) | 24 (20%) | 48 (41%) | 74 (63%) | 93 (79%) |
| Frozen CodeBERT + weighted LR (alternative) | 9 (8%) | 14 (12%) | 33 (28%) | 46 (39%) | 75 (64%) |
| TF-IDF + weighted LR | 17 (14%) | 28 (24%) | 51 (43%) | 72 (61%) | 89 (75%) |
| Code length only | 13 (11%) | 21 (18%) | 31 (26%) | 46 (39%) | 74 (63%) |

### 9.6 Recall by type of weakness

Table 11 comes straight from the warehouse: one SQL query joins predictions, functions and the MITRE catalog. Each CWE has only a handful of test cases, so the table is a hint, not a result. The model finds about half of the injection-type weaknesses (cross-site scripting, XML external entities, path traversal) but

almost none of the access-control ones: 0 of 5 of the CWE-264 cases and 1 of 7 of the CSRF cases. Those depend on configuration and on code outside a single function.

Table 11. Recall of the primary model by CWE on the test split, for CWEs with at least 4 vulnerable test functions. This table is a SQL query on the warehouse (`evaluate.py`).

| CWE | Name | Vulnerable | Caught | Recall |
|---|---|---|---|---|
| CWE-611 | Improper Restriction of XML External Entity Reference | 11 | 5 | 46% |
| CWE-79 | Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') | 9 | 5 | 56% |
| CWE-20 | Improper Input Validation | 7 | 4 | 57% |
| CWE-200 | Exposure of Sensitive Information to an Unauthorized Actor | 7 | 3 | 43% |
| CWE-352 | Cross-Site Request Forgery (CSRF) | 7 | 1 | 14% |
| NVD-CWE-noinfo | Insufficient Information | 7 | 5 | 71% |
| CWE-22 | Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') | 5 | 3 | 60% |
| CWE-264 | Permissions, Privileges, and Access Controls | 5 | 0 | 0% |
| CWE-287 | Improper Authentication | 5 | 3 | 60% |
| CWE-835 | Loop with Unreachable Exit Condition ('Infinite Loop') | 5 | 3 | 60% |
| CWE-502 | Deserialization of Untrusted Data | 4 | 2 | 50% |
| CWE-755 | Improper Handling of Exceptional Conditions | 4 | 4 | 100% |
| CWE-863 | Incorrect Authorization | 4 | 3 | 75% |

## 9.7 The balanced dataset

On the balanced dataset every negative is the fixed version of a vulnerable function, so there are no easy cases. The same method reaches a test ROC AUC of 0.560 here (95% interval 0.490 to 0.634). The interval includes 0.5, so I cannot say the model does better than chance. The paper reports 0.66 for ContraBERT and 0.69 for WizardCoder, and I did not reach their number. Likely reasons: ContraBERT is a stronger relative of CodeBERT, the paper tuned its settings on this dataset while I used standard ones, and the training split has only 801 functions. TF-IDF reaches 0.598, also close to chance. The F1 of 0.641 in Table 12 looks good but means little: the best-F1 threshold flags almost every function, and flagging all of them already gives an F1 of 0.645 on this split.

This is section 9.4 again from another side. Telling a vulnerable function from its own fix is the real task, and none of my models can do it.

Table 12. Test split of X1 without P3 (246 functions, 117 vulnerable).

| Model | Accuracy | Precision | Recall | F1 | ROC AUC (95% interval) |
|---|---|---|---|---|---|
| CodeBERT + class-weighted CE (primary) | 50.0% | 0.487 | 0.940 | 0.641 | 0.560 (0.490–0.634) |
| TF-IDF + weighted LR | 48.0% | 0.477 | 0.974 | 0.640 | 0.598 (0.523–0.670) |
| Code length only | 47.6% | 0.476 | 1.000 | 0.645 | 0.536 (0.464–0.609) |
| Always "not vulnerable" | 52.4% | 0.000 | 0.000 | 0.000 | 0.500 (0.500–0.500) |
| *ContraBERT, reported by Shestov et al.* | – | – | – | – | *0.66* |
| *WizardCoder 13B, reported by Shestov et al.* | – | – | – | – | *0.69* |

## 9.8 Primary method against the alternative

The alternative freezes CodeBERT and trains only 769 numbers. It took 3 minutes in total, almost all of it one pass of CodeBERT over the data, against 49 minutes for the primary method. It is clearly worse:

ROC AUC 0.819 against 0.849 and F1 0.153 against 0.222, both clear differences in Table 9. At a 10% review budget it finds 39% of the vulnerable functions, no more than the length-only model. The frozen vectors hold little that a linear model can use for this task; the fine-tuning is where the primary method gets its edge.

Table 13. Cost against quality on X1 with P3 (test split). "Found" is the share of vulnerable functions in the top 10% of scores.

| Method | Trained numbers | Time | Hardware | ROC AUC | F1 | Found |
|---|---|---|---|---|---|---|
| CodeBERT + class-weighted CE (primary) | 124,647,170 | 49 min | Apple M3 Pro GPU | 0.849 | 0.222 | 53% |
| CodeBERT + plain CE | 124,647,170 | 48 min | Apple M3 Pro GPU | 0.862 | 0.278 | 63% |
| CodeBERT + square-root weights | 124,647,170 | 48 min | Apple M3 Pro GPU | 0.851 | 0.215 | 53% |
| Frozen CodeBERT + weighted LR (alternative) | 769 | 3 min | GPU for the vectors | 0.819 | 0.153 | 39% |
| TF-IDF + weighted LR | 100,001 | 2 s | CPU | 0.871 | 0.326 | 61% |
| Code length only | 2 | 0 s | CPU | 0.807 | 0.165 | 39% |

For an organization the choice depends on what it can run and explain. With a GPU, fine-tuning takes about an hour and gives a model that orders the review queue well. Without one, TF-IDF gives the same ranking quality in 2 seconds on a normal CPU, and its decisions are easier to explain, because each word has a weight. The frozen alternative is the weakest choice: it still needs CodeBERT and a GPU for the vectors, and it ranks no better than code length.

## 10. Recommendations

These follow from the numbers above and are written for a team that wants to use such a model on its own Java code.

1. **Use the model to order the review queue, never to approve code.** Reading the top 10% of functions finds about half of the vulnerable ones; the other half is missed. A function the model does not flag is not safe.
2. **Measure recall at a review budget, not accuracy.** At one vulnerable function in 35, accuracy rewards flagging nothing. Agree on how many functions the team can read, and report how many real vulnerabilities that finds. Check the threshold again on the team's own code, because it shifts between sets of commits (6.8% against 9.7% flagged at the same threshold).
3. **Fix the data before buying a bigger model.** The high scores come from the easy negatives, and on this data the paper's 13-billion-parameter model gained only about 0.01 ROC AUC over the CodeBERT-style ContraBERT. Better data would help more: a split by project, expert review of the 41 quarantined rows and of noisy labels, and a way to read functions longer than 512 tokens.
4. **Pick the cheapest method that is good enough.** On this data TF-IDF ranked functions as well as fine-tuned CodeBERT, in seconds on a CPU instead of about 50 minutes on a GPU. The frozen alternative saves time but ranks no better than code length. A bigger model pays off only once the data is good enough for its extra capacity to matter.
5. **Do not add class weights by default; tune the threshold instead.** In my runs the weights only moved where 0.5 falls and made the probabilities less reliable. Training without weights and choosing the threshold for the team's review budget was at least as good (F1 0.278 against 0.222), and a threshold can be changed later without retraining.

## 11. Limitations

- **One run per setting.** Each CodeBERT setting was trained once, with one seed. Training on the MPS GPU is not bit-for-bit repeatable, so a second run would give slightly different numbers. The bootstrap intervals cover the noise from the small test split, not the noise from training. A difference between two runs that is smaller than these intervals should not be read as real.
- **No split by project.** The dataset has no project field, so the split is by commit, as in the paper. Code from the same project can sit in training and in test. On a project the model has never seen, the scores would most likely be lower. The paper names the same threat.
- **Code is cut at 512 tokens.** About half of the vulnerable functions are longer, so the model sees only their beginning. The paper found that a longer context did not help its model on the balanced set, but I did not test this for CodeBERT.
- **Labels come from a rule.** Nobody checked the labels by hand. The 41 quarantined rows are only the conflicts I could detect; wrong labels without a conflicting copy stay in the data.
- **Not the paper's exact test set.** The published test file has 4,576 rows while the paper reports 4,567, and my cleaning removed 29 more. I compare my scores with the numbers printed in the paper; I did not re-run ContraBERT or WizardCoder.
- **Few hyperparameters tried.** I used common CodeBERT settings and did not tune them, because one run takes about an hour on my laptop. The paper tuned its models. All three CodeBERT runs on X1 with P3 had their best validation score in the last of their four epochs, so longer training might help a little.

## 12. How to run it

```
python3.12 -m venv ~/venvs/mood-midterm && ~/venvs/mood-midterm/bin/pip install -r requirements.txt
PY=~/venvs/mood-midterm/bin/python
$PY data_pipeline.py                                # download, check, clean, load (under a minute)
$PY train.py --run codebert_weighted                # primary method (about 49 min on an M3 Pro)
$PY train.py --run codebert_plain                   # same without class weights; also codebert_sqrt,
$PY train.py --run codebert_weighted_balanced_set   #   codebert_weighted_balanced_set
$PY alternative.py && $PY baselines.py && $PY evaluate.py
$PY demo.py --sample 2                              # live demo
```

The notebook `midterm_project.ipynb` imports the same files and is already executed. The data steps, the baselines, the evaluation and the demo run inside it. Training takes about three hours in total, so the notebook reads the saved training runs from `runs/` unless `RUN_TRAINING` is switched on. Every table and chart in this report comes from `out/`, which that run produced.

## 13. References


1. Shestov, A., Levichev, R., Mussabayev, R., Maslov, E., Zadorozhny, P., Cheshkov, A., Mussabayev, R., Toleu, A., Tolegen, G., Krassovitskiy, A. (2025). Finetuning large language models for vulnerability detection. *IEEE Access*, 13, 38889–38900. doi:10.1109/ACCESS.2025.3546700. Code and data: github.com/rmusab/vul-llm-finetune.
2. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M. (2020). CodeBERT: a pre-trained model for programming and natural languages. *Findings of EMNLP 2020*, 1536–1547.
3. Croft, R., Babar, M. A., Kholoosi, M. M. (2023). Data quality for software vulnerability datasets. *ICSE 2023*, 121–133. doi:10.1109/ICSE48619.2023.00022.
4. Bhandari, G., Naseer, A., Moonen, L. (2021). CVEfixes: automated collection of vulnerabilities and their fixes from open-source software. *PROMISE 2021*, 30–39.

5. Ponta, S. E., Plate, H., Sabetta, A., Bezzi, M., Dangremont, C. (2019). A manually-curated dataset of fixes to vulnerabilities of open-source software. *MSR 2019*, 383–387.
6. Lin, T.-Y., Goyal, P., Girshick, R., He, K., Dollár, P. (2017). Focal loss for dense object detection. *ICCV 2017*, 2999–3007.
7. Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
8. MITRE. Common Weakness Enumeration (CWE), version 4.20. cwe.mitre.org.
9. DAMA International (2017). *DAMA-DMBOK: Data Management Body of Knowledge*, 2nd ed. Technics Publications.
10. ISO 8000 (data quality); ISO/IEC 11179 (metadata registries); ISO/IEC 25012:2008 (data quality model); ISO/IEC 27001:2022 (information security); ISO/IEC 38505-1:2017 (governance of data); ISO/IEC 42001:2023 (AI management systems).
11. NIST (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1.
12. Regulation (EU) 2016/679 (General Data Protection Regulation). Law of the Republic of Kazakhstan No. 94-V of 21 May 2013 "On personal data and their protection".
13. NVD. CVE-2025-32434: remote code execution in `torch.load` with `weights_only=True` before PyTorch 2.6.0.
14. Seralina, N. T. (2026). *Management of Organizational Data*, Lectures 1, 2 and 4. KBTU.